

Verifying Floating-Point Programs in Stainless

Andrea Gilot^{*✉} , Axel Bergström^{*} , and Eva Darulova 

Uppsala University

{andrea.gilot,axel.bergstrom,eva.darulova}@it.uu.se

Abstract. We extend the Stainless deductive verifier with floating-point support, providing the first automated verification support for floating-point numbers for a subset of Scala that includes polymorphism, recursion and higher-order functions. We follow the recent approach in the KeY verifier to axiomatise reasoning about mathematical functions, but go further by supporting all functions from Scala’s math API, and by verifying the correctness of the axioms against the actual implementation in Stainless itself. We validate Stainless’ floating-point support on a new set of benchmarks sampled from real-world code from GitHub, showing that it can verify specifications about, *e.g.*, ranges of output or absence of special values for most supported functions, or produce counter-examples when the specifications do not hold.

Keywords: Floating-point Arithmetic · Transcendental Functions · Deductive Verification

1 Introduction

Floating-point arithmetic is ubiquitous in modern software. While it is often discussed in the context of scientific computing, embedded systems, and machine learning, it is first and foremost pervasive in everyday user code [18]. Yet, verification tools do not always support floating-point reasoning, limiting their use in practice. For example, the absence of floating-point support in Liquid Haskell [40] has been identified as a limitation to its applicability in the real world [39].

This work introduces bit-accurate floating-point reasoning into Stainless [20], a deductive verification framework for Scala programs. Two main challenges in this integration are correctness and performance. To ensure correctness, the implementation must carefully address semantic discrepancies between the JVM and SMT-LIB, and the non-standard behaviour of equality.

Moreover, reasoning about floating-point arithmetic in SMT solvers is known to be expensive, requiring a careful encoding of verification conditions using specialised SMT-LIB constructs when available.

We start by reproducing the floating-point support recently introduced in KeY [1], demonstrating that this approach can be generalised to other verifiers. Our integration extends this support to the subset of the Scala language handled

^{*} These authors contributed equally to this work.

by Stainless, enabling the verification of programs that combine floating-point computations with, among others, higher-order functions and algebraic data types, features unsupported in KeY. We address the challenge of combining polymorphic equality, often assumed to be reflexive, with floating-point reasoning, where this assumption does not hold (because `NaN != NaN`). Ignoring this mismatch can lead to unsoundness, as theorems proven for polymorphic types may be unsafely instantiated with floating-point types.

We implement a standard math library for Stainless that mirrors the Scala standard library API, and verify specifications *in Stainless itself* that were previously only axiomatised in KeY or not supported at all. While these specifications are partial, *e.g.*, encoding that `sin` returns a value between -1 and 1, they are sometimes sufficient for verifying user code with transcendental functions. SMT-solvers do not support transcendental functions, and inlining the implementation of math library functions will typically yield unacceptably long solving times.

To minimise user annotation effort, Stainless *automatically* generates verification conditions that capture subtle aspects of floating-point behaviour that are counter-intuitive and often misunderstood, even among experienced developers [14]. Specifically, it enforces non-`NaN` preconditions for floating-point comparisons and verifies the safety of casts between floating-point and integer types.

We evaluate Stainless on the benchmarks proposed by KeY [1], showing comparable support for floating-point reasoning. We further assess Stainless on 103 real-world floating-point functions mined from open-source GitHub repositories, identifying both practical use cases and remaining challenges—for Stainless and beyond—for achieving comprehensive floating-point verification support.

Our results show that Stainless successfully verifies all benchmarks from KeY and either verifies or produces counterexamples for 82% of user benchmarks. Reasoning about transcendental functions via properties verified in our mathematical library is sufficient for more than half of these user benchmarks, highlighting both its practical effectiveness but also its current limitations. Finally, automated checks for unintuitive floating-point behaviour detect every occurrence of the issues checked for in user code, effectively identifying potential bugs.

Contributions. In summary, we make the following contributions:

- We extend the Stainless verifier with floating-point support¹, the recent SMT-solver Bitwuzla [33] as a backend specialising in floating-point reasoning, and usability features such as automatic checks for unintuitive floating-point behaviour (Section 3).
- We implement a standard math library for Stainless that closely follows the OpenJDK math library implementation and verify mathematical properties of this library with Stainless itself (Section 4).
- We evaluate Stainless on the set of benchmarks proposed by KeY as well as on user-written floating-point code mined on GitHub² (Section 5).

¹ Available at: <https://github.com/epfl-lara/stainless>.

² <https://github.com/fxpl/stainless-float-benchmarks>

```

case class StormDay(moves: Int, errors: Int) {
  require(moves >= errors && errors >= 0 && moves > 0) // class invariant

  def accuracyPercent: Float = {
    100 * (moves - errors) / moves.toFloat
  }.ensuring(result => 0 <= result && result <= 100) // postcondition
}

def gradient(prediction: Double, label: Double): Double = {
  require(prediction.isFinite && label.isFinite) // precondition
  - 4.0 * label / (1.0 + math.exp(2.0 * label * prediction))
}.ensuring(result => !result.isNaN)

def limit(maxMagnitude: Double): Vector = {
  if (getMagnitude > maxMagnitude) // NaN check inserted here
    getNewMagnitude(maxMagnitude)
  else Vector(x, y)
}

```

Fig. 1: Example floating-point programs extracted from GitHub

2 Overview

Stainless did not previously support floating-point reasoning. In this section, we provide an overview of Stainless and of the floating-point support introduced in this work. Stainless is a contract-based verifier for Scala programs [20]. Every Stainless program is valid Scala code and can be compiled and executed on the JVM like any other Scala program. The class `StormDay` in Figure 1 illustrates a fragment of the Lichess codebase, an online chess platform. The function `accuracyPercent` computes a player’s accuracy by counting the number of correct moves in a sequence of chess moves. Under the assumption that the number of moves is strictly positive and greater than the number of errors (expressed in the class invariant in the **require** clause), we can specify in the postcondition (**ensuring** clause) that the result must lie within the interval $[0, 100]$.

During verification, Stainless generates verification conditions that are discharged by SMT solvers. In this case, Stainless reports the counterexample `StormDay(1073741832, 730144766)`, for which the function returns `-2.9586256E-5`, violating the postcondition. This result occurs because the left multiplication uses integer arithmetic, overflows to a negative value, and is then divided. Rewriting the body of `accuracyPercent` to `100 * ((moves - errors) / moves.toFloat)` forces the floating-point division to occur before the multiplication by 100, guaranteeing that the result always lies within the expected range.

The function `gradient` in Figure 1 implements a gradient computation in Apache Spark [3], a Scala framework for large-scale data processing and the most popular Scala library on GitHub. For this example, we specify that the inputs are finite, *i.e.*, they are neither of the floating-point special values $\pm\text{Infinity}$

nor `NaN`. One can obtain infinite floating-point values due to, *e.g.*, overflow and “Not a Number” values due to mathematically invalid operations, *e.g.*, `sqrt(-1.0)`. Stainless attempts to verify that the function never returns `NaN`, which one can view as saying that it does not encounter an error. When run on this snippet, Stainless reports a counterexample in which `label` is assigned a value of very large magnitude (causing overflow when multiplied by 2) and `prediction` is set to 0, leading to `Infinity * 0 = NaN`. Again reassociating the multiplications to the right solves the problem and ensures compliance with the specification, as verified by Stainless.

This example poses two challenges for the verifier: the detection of `NaN` values and the presence of the exponential function. Detecting `NaN` values is difficult for both developers and static analysers because their precise characterisation requires bit-level reasoning. The difficulty increases in the presence of transcendental functions, `math.exp` in this example, that are not natively supported by SMT solvers and whose implementations are typically too complex to be inlined.

Stainless supports reasoning about mathematical functions from the Scala standard library such as trigonometric, exponential or logarithmic functions. Following the approach of KeY [1], these functions are modelled as axiomatised uninterpreted symbols. This abstraction enables Stainless to perform basic reasoning about the occurrence of special values such as `NaN`, without requiring complete semantic modelling. Unlike KeY, we verify the axioms against the actual implementation with Stainless itself—stress-testing its floating-point support as these functions are highly numerical in nature; Section 4 provides more details.

Special-value bugs frequently arise from missing `NaN` checks [13]. For example, NumPy previously contained a bug in its `max` function, which returned an arbitrary number instead of the maximum when `NaN` values were present in the input array. The cause for such bugs is `NaN`’s unintuitive behaviour: any comparison involving `NaN` evaluates to false, including `NaN == NaN`. To prevent such errors from going undetected, Stainless automatically inserts `NaN` checks for all comparisons and equality operations, ensuring that neither operand is `NaN`.

The function `limit` in Figure 1 clamps the magnitude of a 2D vector if it exceeds a given threshold. If the input threshold is `NaN`, the comparison evaluates to false, causing the function to return the original vector, effectively ignoring the invalid input. With the inserted `NaN` checks, Stainless automatically detects such cases and reports a corresponding counterexample. When comparisons with `NaN` are intentional, users can explicitly disable these checks.

Stainless also inserts checks when casting floating-point values to integers that verify that input arguments are in the correct range. In Scala, casting `NaN` yields 0, while out-of-range values are clamped to the maximum or minimum representable 32-bit or 64-bit integer. For 16-bit and 8-bit integers, however, out-of-range values wrap around modulo the target range, introducing a further source of unintended behaviour.

Most of the properties checked for in our examples and evaluation benchmarks involve range analysis, sign determination, and detection of special values. Floating-point equalities, which Stainless can also reason about, are less common.

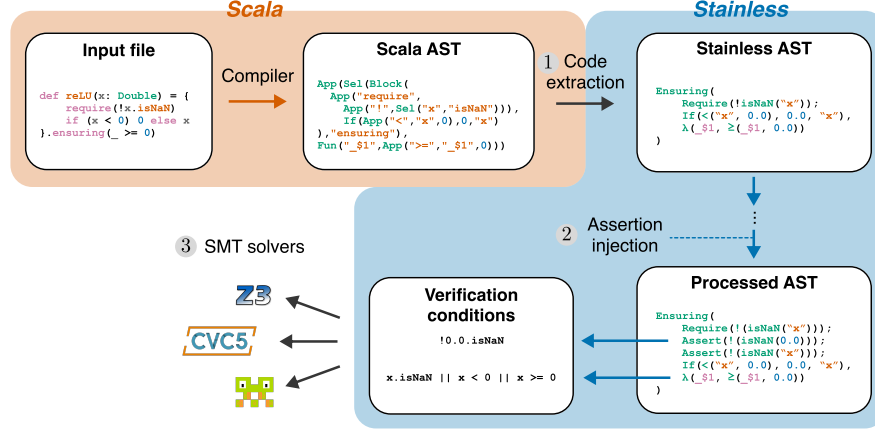


Fig. 2: Simplified architecture of Stainless

This is mainly because expressions that are equal over real numbers (in an ideal specification), will in general not be equal over floating-points due to rounding.

In addition to these high-level floating-point properties, Stainless can also reason about bit-level properties of floating-point values. Our case study in Section 4 requires reasoning about, for example, how bit-level operations on the exponents affect the corresponding floating-point values.

3 Implementation of Floating-Point Support in Stainless

We implement our extension within Stainless, which is implemented in Scala, and reuse existing infrastructure wherever possible. Figure 2 presents an overview of Stainless. Stainless operates on the ASTs produced by the Scala compiler. It then processes the AST in several phases including anti-aliasing, type encoding and termination measure inference. At the end of the pipeline, Stainless generates verification conditions and discharges them to SMT solvers. Note that the Scala compiler preserves floating points semantics (modulo compiler implementation bugs), and does not affect the verification results due to changes in floating-point rounding. Our extension reuses most of the existing infrastructure and brings modifications to only a few phases in the pipeline, thereby showing Stainless infrastructure’s modularity.

We now expand on the modifications we made to Stainless to support floating-point arithmetic both soundly and efficiently that we deem to be the most interesting for other tools and developers.

Code Extraction. (1) Part of the process consists in lowering Scala ASTs into Stainless ASTs. Scala provides high-level constructs, while Stainless uses an intermediate representation tailored for verification, which requires explicit encodings. By default, Stainless inlines non-recursive expressions and represents

primitive operations such as arithmetic, comparisons, and type casts with dedicated nodes. However, SMT-LIB defines more complex operators such as square root, min/max, and rounding (`ceil`, `floor`, `rint`). Mapping Scala calls directly to these operators reduces the size and complexity of verification conditions, so we disable inlining for such functions and translate them directly. For example, Scala implements the floating-point absolute value function by converting the input into binary, applying a mask, and reconstructing a floating-point value. Instead of inlining this bit-level implementation, we map calls to `abs` directly to the corresponding SMT-LIB operator.

Semantic discrepancies between the JVM and SMT-LIB also pose challenges. One such discrepancy concerns the modulo operator: in Scala `14.5 % 1.5` evaluates to `1.0`, while in SMT-LIB it yields `-0.5`. Correcting this difference (e.g., by adding an offset) introduces rounding errors, so Stainless does not currently support floating-point modulo. To assess the impact of this limitation, we analysed 30,421 floating-point functions mined from GitHub (Section 5.1). Only 108 (0.35 %) used modulo, typically with arguments 1 (to extract the fractional part) or 360 (for angle computations). In these cases, if one is willing to accept different floating-point rounding behaviour, one can replace `x % n` with `x - n * floor(x / n)` to avoid the modulo operation (although both expressions are mathematically equivalent over the reals, they accumulate rounding errors differently in floating-point arithmetic). Since Scala runs on the JVM, KeY is subject to the same limitation and likewise omits support for floating-point modulo.

A second discrepancy concerns the representation of `NaN` values. In IEEE 754, `NaN`s are encoded with all exponent bits set and one or more non-zero significand. This yields multiple encodings of `NaN` values where the varying significand bits form a payload. The JVM preserves these payloads, while SMT-LIB uses a single canonical `NaN` value. As a result, functions that expose the binary encoding of floating-point numbers diverge in semantics, since SMT-LIB does not differentiate between different binary `NaN` encodings. To avoid unsoundness, Stainless models binary conversion as an uninterpreted function whose result, when converted back to a floating-point value, equals the original floating-point value.

Equality and Polymorphism. The SMT-LIB standard distinguishes between bitwise equality and IEEE754 equality, where `+0` is equal to `-0` and `NaN` is not equal to anything, including itself. The non-reflexivity of equality is a problem when mixed with parametric polymorphism. In Stainless, reasoning about type variables relies on the assumption that equality is reflexive, symmetric, and transitive. When such type variables are instantiated with floating-point types or with user-defined types that override equality, these assumptions may lead to unsoundness. For instance, Stainless could incorrectly prove `NaN == NaN`.

One possible solution is to drop all assumptions about parametric equality and instead treat it as an uninterpreted symbol, as done in Flux [26]. Although this guarantees soundness, it breaks backward compatibility because many programs that previously verified in Stainless would no longer do so.

We adopt a more pragmatic approach. By default, Stainless only instantiates type variables with types whose equality satisfies reflexivity, symmetry, and

transitivity. If the user wishes to instantiate a polymorphic expression with `Float` or `Double`, they must annotate the type variable with `@noeq`. Stainless then treats equality on the annotated type variable as an uninterpreted predicate, thereby avoiding unsoundness due to assumed algebraic properties of equality.

Even disregarding `NaNs`, IEEE754 equality does not necessarily imply behavioural equivalence either. For example, `+0.0` and `-0.0` compare as equal, even though `.isPositive` returns true for the former and false for the latter. Stainless handles this behaviour correctly.

Assertion Injection. (2) One of the phases we most extensively modified in our work is assertion injection. For integer arithmetic, Stainless inserts checks by default to ensure that operations do not overflow and that divisions by zero do not occur. This behaviour can be disabled globally through command-line options or locally via wrappers. Assertions are injected during AST traversal, depending on the node being processed.

We extend this mechanism with additional checks for floating-point operations. In particular, we add `NaN`-checks on comparisons and safety checks for casts to integers. When processing a comparison node (including equality), Stainless now generates a verification condition ensuring that both operands are non-`NaN`. When traversing a type cast to an integer, Stainless generates two verification conditions: one ensuring that the value is not `NaN`, and another ensuring that the floating-point value lies within the integer domain. As demonstrated in our empirical evaluation (Section 5.4), these checks can detect real bugs in user code.

It could be argued that `NaN` values should be checked globally, *e.g.*, by disallowing them in any function argument. While appealing from a correctness perspective, such control would raise concerns about the usability of the tool. Excessive checks can overwhelm the SMT solver and substantially increase verification time. It is also unclear what assumptions these checks should make about the inputs of checked functions or expressions. In addition, `NaN` values may be acceptable in some contexts (*e.g.*, to describe out-of-domain behaviour). In such cases, users should retain the ability to disable these checks on a per-function basis. Another approach would be to provide fine-grained control, for example by supporting different levels of strictness or configurable options. Evaluating such usability trade-offs requires a user study, which we leave for future work.

Addition of Bitwuzla. (3) We extend the set of available SMT solvers in Stainless to include Bitwuzla [33]. Bitwuzla offers strong native support for floating-point arithmetic and bit-vectors, and outperforms other solvers on these types of queries (Section 5.3). However, it lacks support for algebraic data types (ADTs), which are heavily used in Scala programs. For verification conditions involving ADTs, Stainless falls back to general-purpose solvers such as Z3 [32] or `cvc5` [5].

4 Case Study: Verified Standard Math Library

Floating-point programs often rely on standard mathematical libraries, particularly for transcendental functions [18]. However, implementations of these

functions are currently often too complex for SMT solvers to reason about directly. Instead, the KeY verifier for Java axiomatises them [1], exposing only selected properties such as their output range or behaviour on special input values. We adopt this property-based approach for Stainless, avoiding inlining low-level implementations and keeping verification tractable for programs that depend on them.

Stainless goes further than KeY by verifying that these exposed properties hold for implementations equivalent or close to the OpenJDK math library, which is based on the widely used FdLibm C library [38]. Specifically, we implement a verified mathematical library in Stainless that mirrors the Scala standard library API. Non-transcendental functions use the existing Scala implementation and are inlined during verification. We extend the set of supported transcendental functions beyond those axiomatised in KeY, covering the full Scala math library, including hyperbolic functions, logarithms, cube roots, and the hypotenuse. We also verify additional properties such as sign preservation and relations to the identity function. Table 3 in the appendix summarises and compares the properties supported in KeY and Stainless. Users can choose whether Stainless should reason about transcendental functions through these verified properties or whether it should inline their implementations.

For most transcendental functions (15 out of 18), the implementation in Stainless adapts FdLibm’s code into Single Static Assignment (SSA) form, in which each variable is assigned exactly once. This conversion eliminates mutability and allows us to bypass Stainless’s encoding of mutable state, which would otherwise inflate verification conditions and degrade solver performance. We perform the SSA transformation manually and validate the resulting code by testing it against the original implementation on ten million random inputs, stratified by magnitude, as well as on special values such as zeros, infinities, `NaN`s, and multiples of π . We also oversample subnormal numbers, as they have been shown to be a common source of bugs in floating-point code [13]. As Stainless contracts are compiled by default to runtime assertions, fuzzing the implementations also checks if the proven properties hold for the tested inputs.

Trigonometric Functions. The trigonometric functions `sin`, `cos`, and `tan` are implemented as two distinct steps: 1. a range reduction computing the input modulo $\pi/2$, and 2. the computation of the function value on the reduced argument. Properties of the second step can be verified using the same approach as for the other FdLibm functions. Some more extensive modifications were required to show that the first step returns a value in the expected range, due to poor SMT-solver performance.

The remainder modulo $\pi/2$ (`RemPio2`) operation has cases for infinite or `NaN` values, small inputs, intermediate inputs (magnitude $\sim \leq 2^{20}\pi/2$), and a final case that can handle all finite floating-point numbers. Stainless can easily handle the first two cases, but we did not manage to complete the proofs for the intermediate magnitude case due to poor SMT-solver performance. Instead, we remove the intermediate range case from our implementation, using the algorithm from the

final case also for intermediate inputs. We describe the algorithm for this final case and some of the modification we made for the proof.

It is not possible to compute the remainder using ordinary floating-point operations, since this would yield unacceptably low accuracy. Instead, the algorithm represents values using arrays of either floating-point values or integers to achieve greater precision than ordinary double-precision floats. After converting the input to this form, it is: 1. multiplied by a (input-magnitude-dependent) part of the expansion of $2/\pi$; 2. has the integer part removed; 3. is multiplied by a prefix of the expansion of $\pi/2$; and, 4. is compressed from an array to a double-precision float with a compensation term (another double-precision float containing the part of the result too small in magnitude to be represented by the first one). After our modifications, Stainless can show that this result is in the expected range of $[-\pi/4, \pi/4]$ in about 20 minutes.

Arithmetic operations on these arrays are implemented using loops. Since Stainless is a functional verifier, it does not support loops natively. Instead, Stainless converts loops to recursive functions that return a tuple containing the updated values of all variables modified by the loop. In SMT-LIB, these tuples are encoded as algebraic data types.

This means we cannot rely on the high performance of Bitwuzla, which does not support algebraic data types. Instead, we have to help `cvc5` solve the verification conditions by adding extra assertions and sometimes unrolling a few loop iterations. Both of these are done to improve solver performance. We also modify the conversion of the input to the array form, ensuring that the same array index always represents values of the same magnitude. This simplifies the loop invariants from a disjunction of different cases to a single case.

The original implementation uses an adaptive number of bits of the expansion of $2/\pi$, re-doing part of the computation with more bits if the precision is found to be insufficient. Since it would be very challenging, in Stainless, to show that this process terminates, we simplify the implementation by using a constant number of bits of $2/\pi$.

Finally, we also modified the loop implementing the compression in step 4 to admit a simple invariant. Our implementation of the loop is based on Algorithm 4.1 in Ogita et. al. [35], but uses the `FastTwoSum` function instead of the `TwoSum` function. Together, these modifications allow us to show that the result is in the expected range, and therefore show that the trigonometric functions yield values in the expected ranges.

We tested the trigonometric functions on *all* single-precision floats and 10 billion random double-precision values, in addition to the fuzzing described earlier. Since the implementation is no longer bit-by-bit equivalent to the original, we have to modify the way we test the trigonometric functions. Stainless' implementation of trigonometric functions is within 1 unit in the last place (ulp) of the OpenJDK implementation on all tested inputs.

5 Evaluation

We evaluate our extension of Stainless on three sets of benchmarks to investigate the following research questions:

RQ1: How effectively does Stainless verify the correctness of real-world floating-point programs or generate counter-examples when verification fails?

RQ2: How does the performance of the supported SMT solvers compare on floating-point queries generated from real-world code?

RQ3: Do NaN-checks on comparisons and type cast safety checks detect possible bugs in real-world code?

5.1 Experimental Setup

We use three sets of benchmarks representing real-world floating-point programs: benchmarks used in the evaluation of KeY [1], functions from Stainless’ standard math library (Section 4), and floating-point functions extracted from open-source Scala projects on GitHub.

Throughout this section, we consider each annotated source *file* a single benchmark. Each such benchmark may contain multiple annotated *functions*, each of which may generate multiple verification conditions (VCs).

We run all experiments on Ubuntu 24.02 with an AMD Ryzen 5900X CPU (12 cores at a 3.7 GHz base clock) and 128 GB of memory. We use three SMT-solvers: Z3 [32] (version 4.13.0), cvc5 [5] (version 1.1.2), and Bitwuzla [33] (built from latest GitHub commit [34]). We run all solvers using a single thread.

For the KeY benchmarks, we disable Stainless’s usability-oriented NaN checks and safety checks on type casts, since KeY does not include these features. For the FdLibm-based benchmarks, we also disable these checks to avoid degraded solver performance due to the larger solver context caused by these checks. We keep these checks enabled for the empirical GitHub benchmarks.

KeY Benchmarks. Because Stainless primarily targets functional programs, we adapt the benchmarks to a more functional style. Specifically, we make classes immutable, rewrite mutating methods to return new instances, and replace small fixed-size arrays by tuples. We also merge tightly coupled Java source files into single Scala files, resulting in some Scala benchmarks corresponding to multiple classes or files in the original Java suite.

Unlike Stainless, the KeY verifier supports multiple contracts per method. We translated such KeY benchmarks by (1) adding preconditions common to all contracts as a precondition in Stainless, and (2) encoding each KeY contract as an implication in Stainless, combining them into a single conjunctive postcondition while omitting any duplicated preconditions. If a method in a KeY benchmark has multiple contracts, one of which is invalid, we avoid using the invalid contract to prove other contracts by generating two versions of the method: one containing only valid contracts and one including the invalid contract. No method in the KeY benchmarks contains more than one invalid contract.

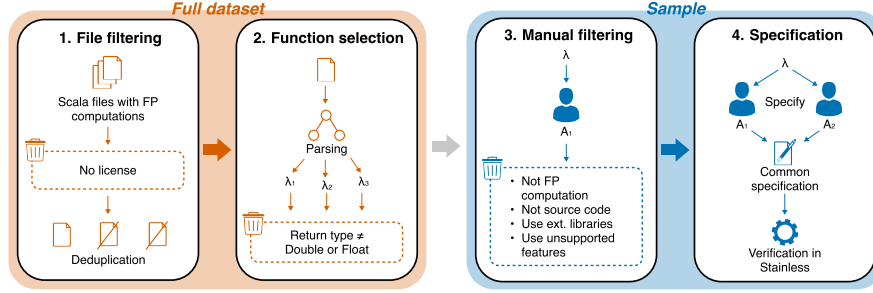


Fig. 3: Overview of the methodology to extract benchmarks from GitHub

FdLibm Benchmarks. The case study presented in Section 4 constitutes the second set of benchmarks. Each function in the math library is converted into one benchmark that requires proving all lemmas about the function, under the assumption that all lemmas about other functions in the library are correct. Two exceptions apply: the sine and cosine functions are combined into a single benchmark since they share most of their code, and the remainder-modulo- $\pi/2$ (RemPio2) operation—used to implement sine, cosine, and tangent functions—is treated as a standalone benchmark.

Empirical Benchmarks. Previous work [18] shows that common floating-point verification benchmarks do not accurately represent real-world programs. In their study, the authors extract a dataset of GitHub source files containing floating-point computations in statically typed languages. We build on this dataset and use the Scala subset to derive empirical benchmarks for Stainless. Figure 3 outlines our overall methodology for extracting and specifying these benchmarks.

We select all Scala files with open-source licenses, remove duplicates to mitigate large-scale code duplication [28], and extract all functions returning floating-point values. From these, we uniformly sample 500 functions and manually exclude:

- 178 functions that return a floating-point value but do not perform any floating-point computation (e.g., simple getters).
- 54 functions that do not belong to the source code of the repository (e.g., to a test suite).
- 97 functions that directly or indirectly call functions from external libraries (e.g., Breeze, Apache Spark).
- 70 functions that use language features unsupported by Stainless, such as macros, concurrency, or bounded polymorphism. When possible, we adapt such functions by removing unsupported features without altering their semantics.

After filtering, we obtain a set of 103 functions from 41 distinct repositories. Since functions may depend on one another, we group functions from the same source file and their dependencies into single verification benchmarks.

The first two authors independently write specifications for these functions, using in-code documentation, usage context, references to published algorithms, and semantic cues (e.g., a variable named `count` is assumed to be non-negative). We systematically examine whether the code can return special values, such as NaN, for valid inputs. The annotators then compare their specifications and resolve disagreements through discussion. When consensus cannot be reached, both specifications are retained and verified independently. If Stainless produces a counterexample when verifying one of the benchmarks, we manually correct the implementation and reattempt verification. This process yields two distinct benchmarks, one representing the original implementation and one incorporating the correction.

The resulting empirical benchmark suite consists of 79 benchmarks from 52 unique source files; 9 source files had unresolved specification disagreements, and 18 benchmarks were derived from fixing counterexamples.

5.2 RQ1: Verification Results on Full Benchmarks

We evaluate the floating-point support in Stainless by running end-to-end experiments on all benchmark sets. We configure Stainless to use `cvc5`, `Bitwuzla`, and `Z3` in a portfolio, with a 300-second timeout per VC for the KeY and the empirical benchmarks. We do not use any timeout for the more numerically intensive FdLibm benchmarks, as they are designed to establish the correctness of specific lemmas about the library functions, which requires all VCs to terminate. We do not empirically compare with KeY or Frama-C (*e.g.*, measuring performance), as these tools target different programming paradigms. Such a comparison would require extensive modification of the benchmarks, as our translation of the KeY benchmarks demonstrates, making the results difficult to interpret.

Three benchmarks are excluded from the evaluation to prevent execution issues. Two of them are empirical benchmark: one causes stack-overflows in Stainless’ termination inference mechanism, while the other one causes Stainless to freeze, requiring manual interruption. We also exclude the FdLibm `RemPio2` benchmark, which triggers out-of-memory errors on the evaluation machine. The benchmark can, however, be successfully verified on the laptop used for the FdLibm case study, where it does not use an unusual amount of memory. We will investigate these issues further.

Table 1 summarises the evaluation results for the KeY and FdLibm benchmarks, and a subset of the empirical benchmarks. Comprehensive statistics for all empirical benchmarks are provided in Table 4 in the appendix. For each benchmark, we report the minimum, median, and maximum Stainless end-to-end wall-clock execution times over five independent runs. We also include the total number of valid and invalid VCs, along with the numbers of VCs reported as unknown or resulting in timeouts. Additionally, for the median run, we report the number of valid and invalid VCs solved by each solver.

For the KeY and FdLibm benchmark sets, all VCs are successfully shown to be either valid or invalid. For the empirical benchmarks, Stainless reports at least one unknown result or timeout in 6 out of 79 empirical cases: 3 benchmarks experience

Table 1: Summary of five end-to-end runs. Min/median/max report end-to-end execution times. #VCs gives the number of verification conditions per benchmark with valid(/invalid/unknown/timeout) results in the run with the median execution time. cvc5, Bitwuzla, and Z3 report the number of VCs proven valid(/invalid) by each solver in that run. #fns shows the number of transcendental function calls (when nonzero).

(a) KeY benchmarks (300s timeout)

benchmark	min	median	max	#vcs	cvc5	Bitwuzla	Z3	#fns
CartesianPolar	12.92	13.05	13.15	3	3	0	0	3
Complex	94.74	95.62	96.06	9/1/0/0	9/1	0	0	3
FPLoop	4.48	4.55	4.67	30/3/0/0	13	12/3	5	
Matrix2	2.50	2.53	2.61	1	1	0	0	
Matrix3	129.44	130.34	130.65	1/1/0/0	1/1	0	0	
PineBench	234.66	245.32	247.01	105/14/0/0	58/2	46/12	1	
Rectangle	10.53	10.55	10.72	1/1/0/0	0/1	0	1	
Rotation	21.05	21.17	21.30	6	4	2	0	

(b) FdLibm benchmarks (no timeout)

benchmark	min	median	max	#VCs	cvc5	Bitwuzla	Z3	#fns
Acosh	33.72	35.60	42.28	11	1	6	4	
Asin	105.46	171.82	859.33	8	1	6	1	
Atan	62.43	64.44	70.88	10	0	8	2	
Atan2	8.40	8.47	8.56	28	0	7	21	2
Cbrt	667.36	913.66	1178.50	11	1	7	3	
Cosh	40.07	41.85	45.40	9	1	7	1	4
Exp	49.85	52.04	152.58	16	0	11	5	
Expm1	3224.93	3969.72	4189.57	24	1	19	4	
Hypot	16.88	17.49	18.24	7	0	6	1	
Log	140.12	206.87	741.37	16	1	14	1	
Log10	4.10	4.21	4.22	6	0	4	2	1
Log1p	22.78	120.52	249.31	16	1	13	2	
Pow	9.40	9.57	9.63	24	1	21	2	
SinCos	194.80	220.15	511.64	42	12	3	27	
Sinh	111.82	119.59	125.88	9	1	8	0	3
Tan	17.80	17.93	18.09	13	3	4	6	
Tanh	4.35	4.42	4.59	6	0	5	1	2

(c) Subset of empirical benchmarks uniformly selected by median time (300s timeout)

benchmark	min	median	max	#VCs	cvc5	Bitwuzla	Z3	#fns
HTMLComp.Position_OK	3.30	3.35	3.43	2	1	1	0	
StreamingIO_CE	3.58	3.60	3.65	2/1/0/0	1/1	1	0	
BoundingBox_a1_OK	3.85	3.89	4.00	4	2	0	2	
AreaUnderCurve_OK	4.16	4.21	4.28	11	10	0	1	
DataHelper_OK	4.69	4.79	4.88	21	4	0	17	
Player_a0_OK	6.26	6.38	6.54	21	16	0	5	
Border_OK	7.74	7.83	7.87	77	14	0	63	
Mediator_a1_OK	9.41	9.50	9.71	56	10	0	46	
Variance_CE	15.25	15.44	15.61	37/2/0/0	18/2	4	15	
Vector2_a0_CE	33.42	33.55	33.75	7/2/0/0	3/2	0	4	
Common_TO	727.42	748.05	916.92	46/0/1/2	15	5	26	1

Table 2: Number of verification conditions solved by each solver

	#VCs	cvc5	Z3	Bitwuzla	Any solver
KeY	176	176 (100%)	150 (85.2%)	118 (67.0%)	176 (100%)
FdLibm	256	227 (88.7%)	200 (78.1%)	217 (84.8%)	254 (99.2%)
Empirical	1600	1595 (99.7%)	1528 (95.5%)	489 (30.6%)	1596 (99.8%)

at least one unknown result, and 4 encounter at least one timeout. All VCs are successfully shown to be either valid or invalid for 73 out of 79 empirical benchmarks.

Manual inspection reveals that for 8 empirical benchmarks detected as invalid by Stainless, the reported counterexamples are spurious.³ These benchmarks contain transcendental functions, which Stainless treats as opaque. That is, only their contract is available to the rest of the codebase and its implementation is hidden. The function call is translated to the SMT solver as an application of an uninterpreted function constrained solely by the contract. If the contract is not strong enough to discharge a verification condition, the solver can construct a model that satisfies the contract while violating the intended semantics of the function, leading to a spurious counterexample. Among the other 11 empirical benchmarks using transcendental functions, one yields a timeout, 5 are verified, and 5 yield non-spurious counterexamples. Overall, the axiomatic approach introduced by KeY is insufficient for 8 out of 19 benchmarks using transcendental functions, *i.e.*, a significant fraction.

Answer to RQ1: Our results indicate that Stainless is usually able to successfully verify or produce counterexamples for self-contained real-world floating-point programs (82 % of empirical cases). In some cases, we observe long verification times with a high variance, especially for the more numerically intensive FdLibm benchmarks. Transcendental functions still pose a challenge: the lemmas introduced in the case study over-approximate their behaviour, leading to spurious counterexamples in several benchmarks (8 out of 19).

5.3 RQ2: Comparison of Solvers on Verification Conditions

To evaluate and compare the performance of the SMT solvers supported by Stainless, we configure Stainless to generate one SMT-LIB file per VC in each benchmark. We run each solver on each generated SMT-LIB file five times and measure the total execution time. For VCs generated from the KeY and FdLibm benchmarks, we use a timeout of 300 seconds. Given the large number of empirical benchmarks, we use a reduced timeout of 120 seconds for this set.

Table 2 reports the number of VCs solved by cvc5, Z3 and Bitwuzla across all benchmark categories. The table also lists the total number of VCs solved by at least one of the three solvers. Compared to the end-to-end evaluation, we

³ For one particular run, the SMT-solvers are not guaranteed to produce the same counterexamples in different runs.

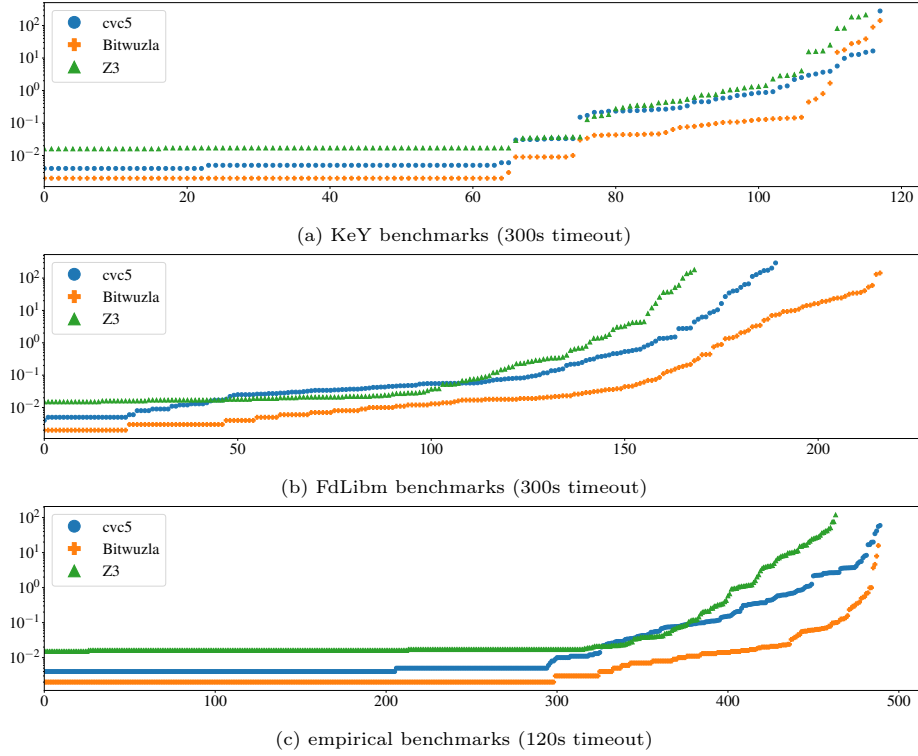


Fig. 4: Solver runtime for verification conditions supported by all solvers

observe fewer unsolved VCs, since unknown results can originate from Stainless itself; in such case we do not observe any unknown results from an SMT-solver. The results indicate that the portfolio approach is effective, with almost all VCs being solved by at least one solver.

Most unsolved VCs for Bitwuzla correspond to errors rather than performance limitations: it yields errors for 58 KeY benchmarks, 37 FdLibm benchmarks, and 1107 empirical benchmarks while *cvc5* and Z3 do not encounter any errors. We believe all of these errors are due to a lack of support for algebraic data types and the theory of integers. We therefore restrict the comparison between the solvers to the subset of VCs where Bitwuzla does not encounter any errors. Figure 4 visualises the performance of the three solvers on this subset, and Figure 5 in the appendix contains corresponding plots for all VCs.

Answer to RQ2: We get a clear ranking of the solvers for the cases where Bitwuzla does not encounter any errors: Bitwuzla performs better than *cvc5*, especially for the numerically intensive FdLibm benchmarks, and *cvc5* performs better than Z3. Since Bitwuzla does not support algebraic data types or the theory of integers, *cvc5* still solves more VCs overall.

5.4 RQ3: Impact of NaN-Checks and Type Cast Safety Checks

Among the 103 functions in our empirical benchmark set, 9 contain floating-point comparisons. In all these cases, proper handling of `NaN` values is missing. In two functions, this omission causes the implementation to return a valid output even when the input is `NaN`, effectively silently ignoring `NaN` inputs. In one case the code comments explicitly assume that a false comparison implies that the input is valid and within a certain range, a property that does not hold when the input is `NaN`; we show this function in Figure 6 in the appendix.

In most of the functions, function arguments are compared against constant thresholds. In such cases, the assumption that inputs are non-`NaN` is sufficient to guarantee the absence of erroneous comparisons. When a function is lacking this assumption, Stainless successfully detects and reports potential `NaN` comparisons.

Floating-point to integer casts occur in two functions within the benchmark set. Both are unsafe, and are reported by Stainless with concrete counterexamples that trigger either `NaN`-to-integer conversions or out-of-bounds integer casts.

Answer to RQ3: Automated `NaN`-checks and type-cast safety checks effectively identify floating-point-related bugs in user code. In our empirical benchmark set, Stainless successfully detects all unsafe type casts and potential `NaN` comparisons, generating concrete counterexamples for each such case.

6 Related Work

To the best of our knowledge, we are the first to introduce floating-point reasoning into a deductive verifier for Scala. Stainless’ floating-point support via verification conditions discharged by SMT solvers using the floating-point SMT-LIB theory [36] follows the support available in KeY for Java, Why3 [15] for C [11] and Ada [16], and the Boogie intermediate language [27]. Compared to these tools, Stainless provides floating-point support for an input language with a different set of features that include polymorphism, recursion, higher-order functions and class invariants. It also adds automated `NaN` checks, reducing the annotation burden compared to existing verifiers. KeY introduced an axiomatic treatment of some transcendental functions from the standard math library [1]. In comparison, Stainless covers the entire standard math library and, for most functions, is able to verify the axioms against the actual implementation itself. For other JVM-based languages, deductive verifiers provide only limited support: VeriFast [24] treats floating-point operations as reals, while OpenJML [10] and LiquidJava [17] can parse floating-point programs but do not reason about them.

Abstract interpretation-based techniques [9, 25] can in principle automatically prove the absence of special values in floating-point code. However, applying these analyses effectively to real-world programs typically requires significant user tuning, such as selecting abstract domains or widening thresholds [7]. Ariadne [6] uses a combination of symbolic execution, real-valued SMT solving and testing to find inputs that trigger floating-point exceptions, including overflow and invalid oper-

ations. Our work subsumes this line of analysis: Stainless not only generates counterexamples through SMT solving, but also proves the absence of such exceptions.

Besides showing the absence of special values, recent research has developed static analyses to bound floating-point rounding errors [12, 19, 29, 31, 37]. These analyses, however, currently target small arithmetic kernels expressed in domain-specific languages, and lack support for features of mainstream programming languages such as user-defined data types, objects, and recursion.

Stainless verifies the axioms for the standard math library functions against the implementations using its own floating-point support. Bagnara et al. [4] verify properties of functions in the `math.h` C library, such as piecewise monotonicity and the absence of NaN results under valid inputs using an alternative approach that combines symbolic execution, abstract interpretation, and exhaustive testing. A complementary line of research verifies the error bounds for floating-point implementations of transcendental functions in interactive theorem provers (*e.g.*, HOL, Rocq) [2, 8, 21–23, 30]; such proofs require an extensive mathematical apparatus and significant user expertise and interaction. Harrison reports that the manual effort required for each such proof can vary from weeks to months [21].

7 Conclusion

Supporting floating-point reasoning in a deductive verifier for programs that use advanced language features such as classes, polymorphism, and higher-order functions requires specific restrictions, for example by limiting instantiations of polymorphic equality or by weakening equality through uninterpreted functions. With our extension of Stainless, we reproduced verification results of KeY and extended them by verifying a large subset of the Scala standard math library, which KeY previously axiomatised or did not support, and showed that Stainless supports verification of user code sampled from GitHub.

Future work includes detecting and eliminating spurious counterexamples arising from transcendental functions. When spurious examples are detected (*e.g.*, by evaluating the program), one possible strategy is to inline the full function implementation and reattempt verification. Current SMT solvers, however, are generally not able to reason efficiently about the complete implementation of transcendental functions, making this approach often ineffective in practice. Strengthening the contracts with additional properties can mitigate the problem, although it does not eliminate the underlying issue completely. Future work also includes investigating usability trade-offs, for example by adding further automated checks and designing more readable specification mechanisms.

Acknowledgments. Co-funded by the European Union (ERC, HORNET, 101163629). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

8 Data Availability Statement

Our floating-point support for Stainless is available in the main branch of the Stainless repository: <https://github.com/epfl-lara/stainless>. We also provide a Zenodo artifact (<https://doi.org/10.5281/zenodo.17486575>) containing the implementation, all benchmarks used in this paper, the collected evaluation results, and scripts to reproduce the experiments.

References

1. Abbasi, R., Schiffel, J., Darulova, E., Ulbrich, M., Ahrendt, W.: Combining Rule- and SMT-based Reasoning for Verifying Floating-Point Java Programs in KeY. *International Journal on Software Tools for Technology Transfer* **25**(2), 185–204 (2023). <https://doi.org/10.1007/s10009-022-00691-x>
2. Akbarpour, B., Abdel-Hamid, A.T., Tahar, S., Harrison, J.: Verifying a Synthesized Implementation of IEEE-754 Floating-Point Exponential Function using HOL. *The Computer Journal* **53**(4), 465–488 (2009). <https://doi.org/10.1093/comjnl/bxp023>
3. Apache Software Foundation: Apache Spark. <https://github.com/apache/spark> (2025), accessed: 2025-10-14
4. Bagnara, R., Chiari, M., Gori, R., Bagnara, A.: A Practical Approach to Verification of Floating-Point C/C++ Programs with math.h/cmath Functions. *ACM Trans. Softw. Eng. Methodol.* **30**(1) (2021). <https://doi.org/10.1145/3410875>
5. Barbosa, H., Barrett, C., Brain, M., Kremer, G., Lachnitt, H., Mann, M., Mohamed, A., Mohamed, M., Niemetz, A., Nötzli, A., Ozdemir, A., Preiner, M., Reynolds, A., Sheng, Y., Tinelli, C., Zohar, Y.: cvc5: A Versatile and Industrial-Strength SMT Solver. In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)* (2022). https://doi.org/10.1007/978-3-030-99524-9_24
6. Barr, E.T., Vo, T., Le, V., Su, Z.: Automatic Detection of Floating-Point Exceptions. In: *Proceedings of the Symposium on Principles of Programming Languages (POPL)* (2013). <https://doi.org/10.1145/2429069.2429133>
7. Blanchet, B., Cousot, P., Cousot, R., Feret, J., Mauborgne, L., Miné, A., Monniaux, D., Rival, X.: A Static Analyzer for Large Safety-Critical Software. In: *Proceedings of the Conference on Programming Language Design and Implementation (PLDI)* (2003). <https://doi.org/10.1145/781131.781153>
8. Boldo, S., Filliâtre, J.C., Melquiond, G.: Combining Coq and Gappa for Certifying Floating-Point Programs. In: *Symposium on the Integration of Symbolic Computation and Mechanized Reasoning (Calculemus)*, Held as part of Conference on Intelligent Computer Mathematics (CICM) (2009). https://doi.org/10.1007/978-3-642-02614-0_10
9. Chen, L., Miné, A., Cousot, P.: A Sound Floating-Point Polyhedra Abstract Domain. In: *Programming Languages and Systems (APLAS)* (2008). https://doi.org/10.1007/978-3-540-89330-1_2
10. Cok, D.R.: OpenJML: JML for Java 7 by Extending OpenJDK. In: *NASA Formal Methods* (2011). https://doi.org/10.1007/978-3-642-20398-5_35
11. Cuoq, P., Kirchner, F., Kosmatov, N., Prevosto, V., Signoles, J., Yakobowski, B.: Frama-C - A Software Analysis Perspective. In: *International Conference on Software Engineering and Formal Methods (SEFM)* (2012). https://doi.org/10.1007/978-3-642-33826-7_16

12. Darulova, E., Izycheva, A., Nasir, F., Ritter, F., Becker, H., Bastian, R.: Daisy - Framework for Analysis and Optimization of Numerical Programs (Tool Paper). In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (2018). https://doi.org/10.1007/978-3-319-89960-2_15
13. Di Franco, A., Guo, H., Rubio-González, C.: A Comprehensive Study of Real-World Numerical Bug Characteristics. In: International Conference on Automated Software Engineering (ASE) (2017). <https://doi.org/10.1109/ASE.2017.8115662>
14. Dinda, P., Hetland, C.: Do Developers Understand IEEE Floating Point? In: International Parallel and Distributed Processing Symposium (IPDPS) (2018). <https://doi.org/10.1109/IPDPS.2018.00068>
15. Filliâtre, J.C., Paskevich, A.: Why3 — Where Programs Meet Provers. In: European Symposium on Programming (ESOP), Programming Languages and Systems (2013). https://doi.org/10.1007/978-3-642-37036-6_8
16. Fumex, C., Marché, C., Moy, Y.: Automating the Verification of Floating-Point Programs. In: Verified Software. Theories, Tools, and Experiments (VSTTE) (2017). https://doi.org/10.1007/978-3-319-72308-2_7
17. Gamboa, C., Canelas, P., Timperley, C., Fonseca, A.: Usability-Oriented Design of Liquid Types for Java. In: International Conference on Software Engineering (ICSE) (2023). <https://doi.org/10.1109/ICSE48619.2023.00132>
18. Gilot, A., Wrigstad, T., Darulova, E.: A Large-Scale Study of Floating-Point Usage in Statically Typed Languages. CoRR **abs/2509.04936** (2025). <https://doi.org/10.48550/ARXIV.2509.04936>
19. Goubault, E., Putot, S.: Static Analysis of Finite Precision Computations. In: Verification, Model Checking, and Abstract Interpretation (VMCAI) (2011). https://doi.org/10.1007/978-3-642-18275-4_17
20. Hamza, J., Voirol, N., Kunčák, V.: System FR: Formalized Foundations for the Stainless Verifier. Proc. ACM Program. Lang. **3**(OOPSLA) (2019). <https://doi.org/10.1145/3360592>
21. Harrison, J.: Floating Point Verification in HOL Light: The Exponential Function. Formal Methods in System Design **16**(3), 271–305 (2000). <https://doi.org/10.1023/A:1008712907154>
22. Harrison, J.: Formal Verification of Floating Point Trigonometric Functions. In: Formal Methods in Computer-Aided Design (FMCAD) (2000). https://doi.org/10.1007/3-540-40922-X_14
23. Harrison, J.: Formal Verification of Square Root Algorithms. Formal Methods in System Design **22**(2), 143–153 (2003). <https://doi.org/10.1023/A:1022973506233>
24. Jacobs, B., Smans, J., Philippaerts, P., Vogels, F., Penninckx, W., Piessens, F.: VeriFast: A Powerful, Sound, Predictable, Fast Verifier for C and Java. In: NASA Formal Methods (2011). https://doi.org/10.1007/978-3-642-20398-5_4
25. Jeannet, B., Miné, A.: Apron: A Library of Numerical Abstract Domains for Static Analysis. In: Computer Aided Verification (CAV) (2009). https://doi.org/10.1007/978-3-642-02658-4_52
26. Lehmann, N., Geller, A.T., Vazou, N., Jhala, R.: Flux: Liquid Types for Rust. Proc. ACM Program. Lang. **7**(PLDI) (2023). <https://doi.org/10.1145/3591283>
27. Leino, K.R.M.: This is Boogie 2. <https://www.microsoft.com/en-us/research/publication/this-is-boogie-2-2/> (2008), accessed: 2025-10-14
28. Lopes, C.V., Maj, P., Martins, P., Saini, V., Yang, D., Zitny, J., Sajnani, H., Vitek, J.: DéjàVu: A Map of Code Duplicates on GitHub. Proc. ACM Program. Lang. **1**(OOPSLA) (2017). <https://doi.org/10.1145/3133908>

29. Magron, V., Constantinides, G., Donaldson, A.: Certified Roundoff Error Bounds Using Semidefinite Programming. *ACM Trans. Math. Softw.* **43**(4) (2017). <https://doi.org/10.1145/3015465>
30. Melquiond, G.: Floating-point arithmetic in the Coq system. *Information and Computation* **216**, 14–23 (2012). <https://doi.org/10.1016/j.ic.2011.09.005>, special Issue: 8th Conference on Real Numbers and Computers
31. Moscato, M., Titolo, L., Dutle, A., Muñoz, C.A.: Automatic Estimation of Verified Floating-Point Round-Off Errors via Static Analysis. In: *Computer Safety, Reliability, and Security (SAFECOMP)* (2017). https://doi.org/10.1007/978-3-319-66266-4_14
32. de Moura, L., Bjørner, N.: Z3: An Efficient SMT Solver. In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)* (2008). https://doi.org/10.1007/978-3-540-78800-3_24
33. Niemetz, A., Preiner, M.: Bitwuzla. In: *Computer Aided Verification (CAV)* (2023). https://doi.org/10.1007/978-3-031-37703-7_1
34. Niemetz, A., Preiner, M.: Bitwuzla public repository - commit used for experiments. <https://github.com/bitwuzla/bitwuzla/tree/929af621f6fe65c78236df70b8eb9343a5914d9f> (2025), accessed: 2025-10-14
35. Ogita, T., Rump, S.M., Oishi, S.: Accurate sum and dot product. *SIAM journal on scientific computing* **26**(6), 1955–1988 (2005). <https://doi.org/10.1137/030601818>
36. Rümmer, P., Wahl, T.: An SMT-LIB theory of binary floating-point arithmetic. In: *International Workshop on Satisfiability Modulo Theories (SMT)* (2010)
37. Solovyev, A., Baranowski, M.S., Briggs, I., Jacobsen, C., Rakamarić, Z., Gopalakrishnan, G.: Rigorous Estimation of Floating-Point Round-Off Errors with Symbolic Taylor Expansions. *ACM Trans. Program. Lang. Syst.* **41**(1) (2018). <https://doi.org/10.1145/3230733>
38. Sun Microsystems Inc.: FDLIBM Version 5.3. <https://www.netlib.org/fdlibm/> (2022), accessed: 2025-10-14
39. Vazou, N., Seidel, E.L., Jhala, R.: LiquidHaskell: Experience with Refinement Types in the Real World. In: *Symposium on Haskell* (2014). <https://doi.org/10.1145/2633357.2633366>
40. Vazou, N., Seidel, E.L., Jhala, R., Vytiniotis, D., Peyton-Jones, S.: Refinement Types for Haskell. In: *International Conference on Functional Programming (ICFP)* (2014). <https://doi.org/10.1145/2628136.2628161>

Appendix

Table 3: Comparison of transcendental function axioms in KeY and verified properties in Stainless

Function	KeY (Axioms)	Stainless (Verified)
cos	NaN, range	NaN, range
sin	NaN, $f(\pm 0)$, range	NaN, $f(\pm 0)$, range
tan	$f(\text{NaN})$, $f(\pm 0)$, range	$f(\text{NaN})$, $f(\pm 0)$, range
asin	$f(\text{NaN})$, $f(0)$, range	NaN, $f(\pm 0)$, $f(\pm 1)$, range
acos	$f(\text{NaN})$, $f(0)$, range	NaN, $f(\pm 0)$, $f(\pm 1)$
atan	$f(\text{NaN})$, $f(0)$, range	NaN, $f(\pm 0)$, $f(\pm \infty)$, sign, range
atan2	$f(\text{NaN}, _)$, $f(_, \text{NaN})$, range	NaN, all pairs of special values, sign, range
hypot	Not supported	NaN, all pairs of special values, range
cbrt	Not supported	NaN, $f(\pm 0)$, $f(\pm 1)$, $f(\pm \infty)$, sign, $f(x) \leq x$
pow	All pairs of special values	All pairs of special values
exp	$f(\text{NaN})$, $f(\pm \infty)$	NaN, $f(\pm 0)$, $f(\pm \infty)$, range
expm1	Not supported	NaN, $f(\pm 0)$, $f(\pm \infty)$, range
log	Not supported	NaN, $f(\pm 0)$, $f(1)$, $f(\pm \infty)$, range, $f(x) \leq x - 1$
log1p	Not supported	NaN, $f(\pm 0)$, $f(-1)$, $f(\pm \infty)$, range, $f(x) \leq x$
log10	Not supported	NaN, $f(\pm 0)$, $f(1)$, $f(\pm \infty)$, range
sinh	Not supported	NaN, $f(\pm 0)$, $f(\pm \infty)$, sign
cosh	Not supported	NaN, $f(\pm 0)$, $f(\pm \infty)$, range
tanh	Not supported	NaN, $f(\pm 0)$, $f(\pm \infty)$, sign, range

Legend:

NaN: exact characterisation of when the output of the function is NaN

$f(\square)$: expected result for input $\square$

all pairs of special values: expected result for all combinations of NaN, ± 0 , and $\pm \infty$

sign: characterisation of the sign of the result

range: lower and/or upper bounds on the result

$f(x) \leq x$: characterisation of when the function is above or below its input

Table 4: Summary statistics for five runs of the end-to-end experiments on the empirical benchmarks. The min/median/max columns summarise the end-to-end execution times. The column `#vcs` shows the total number of verification conditions shown to be valid in the median run, or, if some were not shown to be valid, the number of valid/invalid/unknown/timeout results. The columns `cvc5`, `Bitwuzla`, and `Z3` shows the number of verification conditions shown to be valid or the number shown to be valid/invalid by each solver in the median run. The column `#fns` shows the number of calls to transcendental math library functions, when not zero.

benchmark	min	median	max	#vcs	cvc5	Bitwuzla	Z3	#fns
AbsoluteError_OK	3.35	3.37	3.41	2	0	2	0	
AreaUnderCurve_CE	4.18	4.22	4.38	9/1/0/0	7/1	0	2	
AreaUnderCurve_OK	4.16	4.21	4.28	11	10	0	1	
Border_OK	7.74	7.83	7.87	77	14	0	63	
BoundingBox_a0_OK	4.01	4.07	4.11	6	4	0	2	
BoundingBox_a1_OK	3.85	3.89	4.00	4	2	0	2	
Cast_OK	3.57	3.59	3.63	4	0	4	0	
Common_TO	727.42	748.05	916.92	46/0/1/2	15	5	26	1
DataHelper_OK	4.69	4.79	4.88	21	4	0	17	
DataSetCost_false_CE	32.33	32.47	32.71	12/2/0/0	2/2	0	10	1
Exp.Mutagen_false_CE	16.97	17.13	17.30	11/2/0/0	2/2	0	9	1
FlinkStatistic_OK	4.24	4.29	4.34	10	5	0	5	
Float16_a0_OK	4.27	4.34	4.38	17	7	0	10	
Float16_a1_OK	4.50	4.58	4.62	21	16	2	3	
FollowTargetRule_CE	100.47	100.71	100.76	15/9/0/0	10/9	1	4	4
FollowTargetRule_OK	104.89	104.91	105.39	32/3/0/0	20/3	2	10	4
GCode_a0_CE	3.86	3.88	3.95	2/1/0/0	0/1	1	1	
GCode_a0_OK	3.92	3.94	4.00	4	1	1	2	
GCode_a1_OK	3.80	3.90	4.00	3	1	0	2	
HTMLComp.Position_OK	3.30	3.35	3.43	2	1	1	0	
HyperLogLogPlusPlus_CE	46.40	46.77	46.88	4/4/0/0	2/4	0	2	2
Intro_OK	3.40	3.51	3.53	2	0	2	0	
IsotonicRegression_CE	5.02	5.11	5.68	15/1/0/0	1	0/1	14	
IsotonicRegression_OK	6.43	6.70	7.53	31	5	11	15	
JaccardSimilarity_OK	10.48	10.63	10.78	16	12	0	4	
JaccardSimilarity_UNK	104.82	109.56	428.60	10/0/2/0	6	0	4	
Kitchen_OK	4.70	4.80	4.98	5	1	0	4	
Kmath_a0_OK	8.04	8.25	8.48	12	4	5	3	3
Kmath_a1_OK	8.68	8.77	8.82	12	4	5	3	3
LDABOptimizer_false_CE	6.68	6.77	6.89	3/2/0/0	0/2	0	3	1
List_OK	6.95	7.03	7.21	18	4	0	14	
LogLoss_CE	6.90	6.99	7.10	24/2/0/0	1	8/2	15	6
LogLoss_OK	6.69	6.83	6.87	26	2	9	15	6
MCOpt.DeepStructure_OK	3.60	3.68	3.71	4	4	0	0	
Mediator_a0_OK	4.83	4.89	5.00	13	4	0	9	
Mediator_a0_UNK	6.72	6.91	7.03	7/0/3/0	1	0	6	
Mediator_a1_OK	9.41	9.50	9.71	56	10	0	46	
NewtonSquare_CE	54.00	54.47	54.64	19/3/0/0	6	13/3	0	
OntologyStatistics_a0_OK	3.51	3.56	3.68	5	1	4	0	
OntologyStatistics_a1_OK	3.73	3.84	3.91	2	0	2	0	
Optimizer_OK	3.67	3.71	3.79	7	1	1	5	
PairwiseScorer_OK	5.39	5.49	5.56	18	6	0	12	
Player_a0_OK	6.26	6.38	6.54	21	16	0	5	

Player_a1_OK	7.65	7.87	8.03	29	26	0	3	
Polyn.Mutagen_false_CE	14.29	14.37	14.53	9/2/0/0	1/2	0	8	2
PolynomialPerformance_CE	4.33	4.38	4.40	12/2/0/0	0	4/2	8	
PolynomialPerformance_TO	430.95	432.50	435.50	16/0/0/1	1	6	9	
PredictionMetrics_CE	19.13	19.14	19.29	11/2/0/0	2/2	4	5	2
PredictionMetrics_OK	15.69	15.83	16.10	17	4	0	13	2
Profile_CE	3.99	4.11	4.19	4/3/0/0	3/1	0/2	1	
Profile_OK	4.03	4.10	4.29	9	4	1	4	
Rectangle_OK	28.62	28.70	28.81	12	11	0	1	
RectangularBorder_a0_OK	11.86	12.28	12.34	30	12	8	10	
RectangularBorder_a1_OK	7.90	8.02	8.07	27	15	6	6	
RedisStore_OK	3.53	3.59	3.65	4	2	2	0	
RegularPayGrantCalc._OK	4.39	4.43	4.51	14	2	9	3	
SamplingUtils_CE	12.52	12.80	13.76	46/2/0/0	11	33/2	2	2
SamplingUtils_false_CE	13.56	14.02	14.62	50/1/0/0	19	29/1	2	2
Session_CE	30.77	33.12	35.06	21/2/0/0	6	15/2	0	
SpeedControl_OK	5.56	5.61	5.76	65	24	41	0	
Spherical_false_CE	21.40	23.21	23.29	73/11/0/0	21	46/11	6	29
StatCounter_OK	318.74	318.82	318.93	19/0/0/1	8	0	11	
StatCounter_TO	318.73	318.81	319.04	17/1/0/1	6/1	0	11	
StormDay_CE	3.95	3.98	4.02	2/3/0/0	0/2	0	2/1	
StormDay_OK	4.81	4.92	4.97	7	3	0	4	
StreamingHistogram_CE	8.68	8.80	8.95	10/1/0/0	1/1	0	9	
StreamingHistogram_OK	8.78	8.85	9.53	15	5	0	10	
StreamingIO_CE	3.58	3.60	3.65	2/1/0/0	1/1	1	0	
StreamingIO_OK	3.38	3.47	3.54	3	1	2	0	
Variance_CE	15.25	15.44	15.61	37/2/0/0	18/2	4	15	
Variance_OK	13.81	13.93	14.12	43	19	4	20	
Vector2_a0_CE	33.42	33.55	33.75	7/2/0/0	3/2	0	4	
Vector2_a0_OK	33.64	33.72	34.12	16	8	3	5	
Vector2_a1_CE	34.43	34.67	34.77	6/3/0/0	2/3	1	3	
Vector2_a1_false_CE	6.08	6.12	6.23	18/1/0/0	6/1	0	12	1
Vectors_false_CE	42.87	43.45	44.02	201/3/0/0	64/1	0	137/2	2
WeightedLeastSquares_OK	15.87	15.90	16.03	17	7	0	10	
gui_OK	3.51	3.63	3.65	3	3	0	0	
package_OK	3.73	3.79	3.80	6	6	0	0	

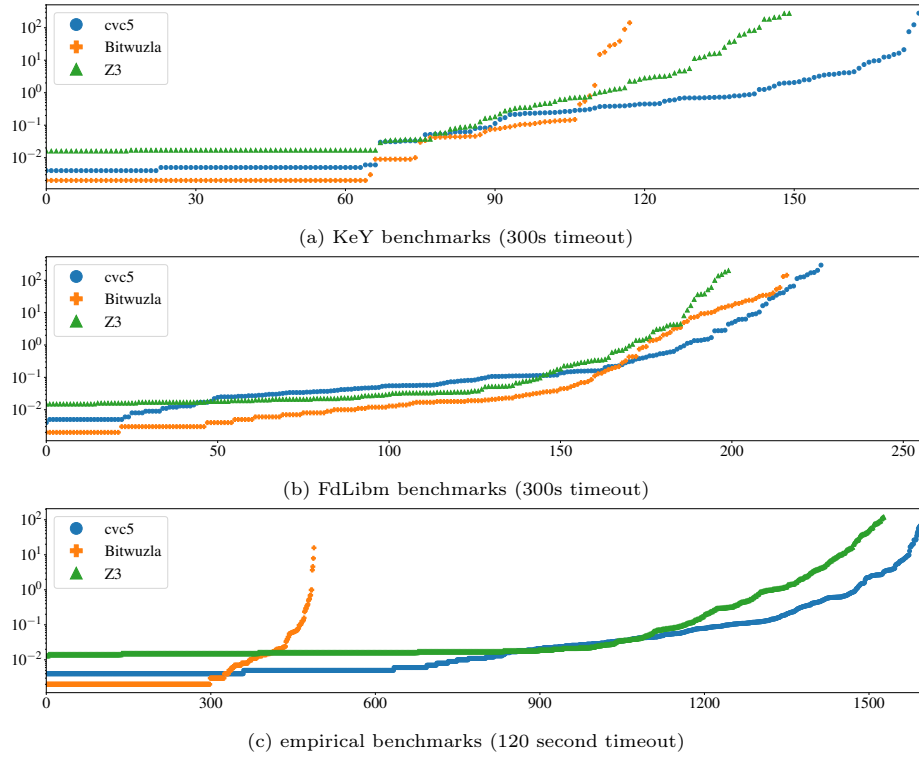


Fig. 5: Solver runtime for all verification conditions

```
def getPortionOutsideBorder(position: Double, minReachablePosition: Double,
    maxReachablePosition: Double): Double = {
    position match {
        case p if p < minReachablePosition => minReachablePosition - p
        case p if p > maxReachablePosition => p - maxReachablePosition
        case _ => 0 // the entity is inside the map edge => radius portion
                       outside the map is 0
    }
}
```

Fig. 6: Example of user code with floating-point comparisons where an in-code comment incorrectly assumes that a false comparison implies a valid (non-NaN) input